In [56]:
```
import numpy as np
```

In [57]:
```
x=np.array(([2,9],[1,5],[3,6]),dtype=float)
y=np.array(([92],[86],[89]),dtype=float)
```

In [69]:
```
x=x/np.amax(x,axis=0)
y=y/100
print(x)
print(y)
```

```
[[0. 0.]
 [0. 1.]
 [1. 0.]
 [1. 1.]]
[[0.  ]
 [0.01]
 [0.01]
 [0.  ]]
```

In [70]:
```
class NeuralNetwork():
    def __init__(self):
        self.inputSize = 2
        self.hiddenSize = 3
        self.outputSize = 1
        self.w1 = np.random.randn(self.inputSize, self.hiddenSize)
        self.w2 = np.random.randn(self.hiddenSize, self.outputSize)

    def feedForward(self, x):
        self.z = np.dot(x, self.w1)
        self.z2 = self.sigmoid(self.z)
        self.z3 = np.dot(self.z2, self.w2)
        output = self.sigmoid(self.z3)
        return output

    def sigmoid(self, s, deriv=False):
        if deriv == True:
            return s * (1 - s)
        return 1 / (1 + np.exp(-s))

    def backward(self, x, y, output):
        self.output_error = y - output
        self.output_delta = self.output_error * self.sigmoid(output, deriv=True)
        self.z2_error = self.output_delta.dot(self.w2.T)
        self.z2_delta = self.z2_error * self.sigmoid(self.z2, deriv=True)
        self.w1 += x.T.dot(self.z2_delta)
        self.w2 += self.z2.T.dot(self.output_delta)

    def train(self, x, y):
        output = self.feedForward(x)
        self.backward(x, y, output)
x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

N = NeuralNetwork()
for i in range(1000):
    N.train(x, y)

print("predicted Output:\n", N.feedForward(x))
```

```
predicted Output:
 [[0.40940811]
 [0.73143211]
 [0.77905046]
 [0.20479363]]
```

In [ ]: